

REPORTE DE ANÁLISIS DE SEGURIDAD

API Visio Flow ? Sistema de Conteo de Personas

Fecha del análisis: 2026-07-09 15:11

Tecnologías evaluadas:

FastAPI (Python) | Flask (Python) | PostgreSQL | Redis | Docker Compose

Estándar aplicado: OWASP API Security Top 10 ? 2023

Clasificación: Académico / Uso interno

RESUMEN EJECUTIVO

10

CRÍTICAS

7

MEDIAS

17

TOTAL

SERVICIOS ANALIZADOS

? FastAPI :8000

API central ? autenticación, áreas, eventos, heatmap, dashboard, usuarios

? Flask :5000

Servicio de visión computacional ? YOLOv8, captura MJPEG, tracking

? Laravel :8085

Dashboard web ? consume API FastAPI vía HTTP interno

? PostgreSQL

Base de datos principal ? 5 tablas: areas, events, state, heatmap, users

? Redis

Caché de sesiones Laravel ? sin autenticación previa al fix

1. INTRODUCCIÓN Y METODOLOGÍA

1.1 Objetivo

Identificar vulnerabilidades de seguridad en el sistema Visio Flow, implementar correcciones directamente en el código fuente, y documentar el proceso con evidencia técnica.

1.2 Alcance

- ? API FastAPI (puerto 8000): 7 routers, 25+ endpoints
- ? Flask Vision Service (puerto 5000): 5 rutas + autenticación por sesión
- ? Infraestructura Docker Compose: 5 servicios
- ? Base de datos PostgreSQL: esquema y configuración
- ? Redis: configuración de caché y autenticación

1.3 Metodología

Se siguió el estándar OWASP API Security Top 10 (2023) como marco de referencia. El análisis se realizó mediante revisión estática del código fuente (SAST manual), inspeccionando cada archivo de configuración, router, servicio y esquema de datos.

Fases del análisis:

1. Reconocimiento ? listado de endpoints y servicios
2. Clasificación ? categorización por riesgo (CRÍTICA / MEDIA)
3. Explotabilidad ? evaluación del impacto potencial
4. Remediación ? implementación de controles de seguridad
5. Verificación ? revisión post-cambio del código corregido

1.4 Herramientas utilizadas

- ? Revisión manual de código fuente (Python, YAML, SQL)
- ? OWASP API Security Top 10 2023 como lista de referencia
- ? python-jose (JWT HS256) para implementación de tokens
- ? slowapi para rate limiting
- ? passlib + bcrypt para hashing (ya existía, validado correcto)

2. HALLAZGOS ? RESUMEN POR SEVERIDAD

ID	SEV	TÍTULO	COMPONENTE
VF-001	CRÍTICA	Autenticación basada en header HTTP falsif...	auth.py
VF-002	CRÍTICA	Login no genera token de sesión seguro (si...	auth.py
VF-003	CRÍTICA	Endpoint POST /events sin autenticación	events.py
VF-004	CRÍTICA	Endpoints POST/GET /heatmap sin autenticación	heatmap.py
VF-005	CRÍTICA	Endpoints GET /state sin autenticación	state.py
VF-006	CRÍTICA	4 endpoints de dashboard sin autenticación	dashboard.py
VF-007	CRÍTICA	Credenciales de DB en texto plano (passwor...	docker-compose.yml
VF-008	CRÍTICA	Redis sin contraseña y puerto 6379 expuest...	docker-compose.yml
VF-009	CRÍTICA	Endpoint /db-test expuesto públicamente	main.py
VF-010	CRÍTICA	Flask secret_key hardcodeada como fallback...	__init__.py
VF-011	MEDIA	Documentación /docs y /redoc expuesta en p...	main.py
VF-012	MEDIA	Sin rate limiting en ningún endpoint	main.py
VF-013	MEDIA	Sin configuración de CORS	main.py
VF-014	MEDIA	Contraseñas débiles permitidas (min_length=1)	auth.py
VF-015	MEDIA	Puerto de PostgreSQL expuesto al host (0.0...	docker-compose.yml
VF-016	MEDIA	APP_KEY de Laravel hardcodeada en docker-c...	docker-compose.yml
VF-017	MEDIA	APP_DEBUG=true en producción (Laravel)	Flask

3. HALLAZGOS DETALLADOS (CRÍTICAS)

VF-001 ? Autenticación basada en header HTTP falsificable

CRÍTICA

OWASP: API2:2023 ? Broken Authentication

Componente: API/app/security/auth.py

ANTES (vulnerable):

```
verify_admin(x_is_admin: bool = Header(False, alias='X-Is-Admin'))
```

DESPUÉS (corregido):

```
get_current_user(token) usando JWT Bearer ? python-jose HS256
```

Impacto: Cualquier cliente podía enviar X-Is-Admin: true y obtener acceso admin completo.

Fix aplicado: Reemplazado por verificación criptográfica de JWT firmado con HS256.

VF-002 ? Login no genera token de sesión seguro (sin JWT)

CRÍTICA

OWASP: API2:2023 ? Broken Authentication

Componente: API/app/router/auth.py

ANTES (vulnerable):

```
LoginResponse(user=UserOut) ? solo retornaba datos del usuario
```

DESPUÉS (corregido):

```
TokenOut(access_token=JWT, token_type='bearer', user=UserOut)
```

Impacto: Sin token, cualquier endpoint 'protegido' era bypassable con el header X-Is-Admin.

Fix aplicado: Login ahora genera JWT HS256 con subject=user_id, is_admin, exp.

VF-003 ? Endpoint POST /events sin autenticación

CRÍTICA

OWASP: API1:2023 ? Broken Object Level Authorization

Componente: API/app/router/events.py

ANTES (vulnerable):

```
def ingest_event(payload, db) ? sin dependencia de auth
```

DESPUÉS (corregido):

```
dependencies=[Depends(get_current_user)]
```

Impacto: Cualquier agente externo podía insertar eventos falsos en la base de datos.

Fix aplicado: Dependencia JWT get_current_user agregada al endpoint.

3. HALLAZGOS DETALLADOS (CRÍTICAS)

VF-004 ? Endpoints POST/GET /heatmap sin autenticación	CRÍTICA
OWASP: API1:2023 ? Broken Object Level Authorization Componente: API/app/router/heatmap.py ANTES (vulnerable): <pre>@router.post("") y @router.get("") sin auth</pre> DESPUÉS (corregido): <pre>dependencies=[Depends(get_current_user)] en ambos endpoints</pre> Impacto: Lectura y escritura de datos de heatmap sin restricción. Fix aplicado: Ambos endpoints protegidos con JWT.	
VF-005 ? Endpoints GET /state sin autenticación	CRÍTICA
OWASP: API1:2023 ? Broken Object Level Authorization Componente: API/app/router/state.py ANTES (vulnerable): <pre>@router.get("") y @router.get("/{area_id}") sin auth</pre> DESPUÉS (corregido): <pre>dependencies=[Depends(get_current_user)] en ambos</pre> Impacto: Estado en tiempo real de personas por área expuesto públicamente. Fix aplicado: Endpoints protegidos con JWT.	
VF-006 ? 4 endpoints de dashboard sin autenticación	CRÍTICA
OWASP: API1:2023 ? Broken Object Level Authorization Componente: API/app/router/dashboard.py ANTES (vulnerable): <pre>/dashboard/summary, /areas, /events, /heatmap ? sin auth</pre> DESPUÉS (corregido): <pre>dependencies=[Depends(get_current_user)] en los 4 endpoints</pre> Impacto: Datos analíticos sensibles accesibles sin credenciales. Fix aplicado: Los 4 endpoints del dashboard protegidos con JWT.	

3. HALLAZGOS DETALLADOS (CRÍTICAS)

VF-007 ? Credenciales de DB en texto plano (password: '123456')	CRÍTICA
OWASP: API8:2023 ? Security Misconfiguration Componente: docker-compose.yml ANTES (vulnerable): <pre>POSTGRES_PASSWORD: '123456' hardcodeada en docker-compose.yml</pre> DESPUÉS (corregido): <pre>POSTGRES_PASSWORD: \${POSTGRES_PASSWORD} desde archivo .env</pre> Impacto: Credenciales expuestas en control de versiones y a cualquier persona con acceso al archivo. Fix aplicado: Externalizadas a .env con .env.example como plantilla documentada.	
VF-008 ? Redis sin contraseña y puerto 6379 expuesto al host	CRÍTICA
OWASP: API8:2023 ? Security Misconfiguration Componente: docker-compose.yml ANTES (vulnerable): <pre>redis: image: redis:alpine, ports: - '6379:6379' (sin auth)</pre> DESPUÉS (corregido): <pre>command: redis-server --requirepass \${REDIS_PASSWORD}, sin puerto expuesto</pre> Impacto: Cualquier proceso en el host podía leer/escribir en Redis (sesiones, caché). Fix aplicado: Redis con contraseña y puerto eliminado de la exposición al host.	
VF-009 ? Endpoint /db-test expuesto públicamente	CRÍTICA
OWASP: API8:2023 ? Security Misconfiguration Componente: API/app/main.py ANTES (vulnerable): <pre>@app.get('/db-test') retorna estado de la DB sin auth</pre> DESPUÉS (corregido): <pre>Endpoint eliminado del código de producción</pre> Impacto: Expone confirmación de conectividad y posibles detalles de error de la DB. Fix aplicado: Endpoint /db-test eliminado completamente.	

3. HALLAZGOS DETALLADOS (CRÍTICAS)

VF-010 ? Flask secret_key hardcodeda como fallback inseguro	CRÍTICA
OWASP: API8:2023 ? Security Misconfiguration Componente: contador_personas_flask/app/__init__.py ANTES (vulnerable): <pre>app.secret_key = os.getenv('FLASK_SECRET_KEY', 'super-secret-key-123')</pre> DESPUÉS (corregido): <pre>Fuerza FLASK_SECRET_KEY desde env; error fatal si no está definida</pre> Impacto: Sesiones podían ser forjadas conociendo el valor 'super-secret-key-123'. Fix aplicado: Secret key sin fallback + SESSION_COOKIE_HTTPONLY + SESSION_COOKIE_SAMESITE=Lax.	
VF-011 ? Documentación /docs y /redoc expuesta en producción	MEDIA
OWASP: API8:2023 ? Security Misconfiguration Componente: API/app/main.py ANTES (vulnerable): <pre>FastAPI() sin restricción de docs_url/redoc_url</pre> DESPUÉS (corregido): <pre>docs_url='/docs' if IS_DEV else None (controlado por APP_ENV)</pre> Impacto: Expone el schema completo de la API a atacantes para reconocimiento. Fix aplicado: Docs deshabilitados cuando APP_ENV != 'development'.	
VF-012 ? Sin rate limiting en ningún endpoint	MEDIA
OWASP: API4:2023 ? Unrestricted Resource Consumption Componente: API/app/main.py ANTES (vulnerable): <pre>Sin middleware de limitación de peticiones</pre> DESPUÉS (corregido): <pre>slowapi Limiter con límite por defecto de 100 req/min por IP</pre> Impacto: Vulnerabilidad a ataques de fuerza bruta en /auth/login y DoS. Fix aplicado: Integrado slowapi con get_remote_address como key_func.	

3. HALLAZGOS DETALLADOS (MEDIAS)

VF-013 ? Sin configuración de CORS	MEDIA
OWASP: API8:2023 ? Security Misconfiguration Componente: API/app/main.py ANTES (vulnerable): <pre>Sin CORSMiddleware ? responde a cualquier origen</pre> DESPUÉS (corregido): <pre>CORSMiddleware con allow_origins desde CORS_ALLOWED_ORIGINS env var</pre> Impacto: Permite que páginas web maliciosas realicen requests a la API. Fix aplicado: CORS configurado con lista explícita de orígenes permitidos.	
VF-014 ? Contraseñas débiles permitidas (min_length=1)	MEDIA
OWASP: API2:2023 ? Broken Authentication Componente: API/app/schemas/auth.py ANTES (vulnerable): <pre>password: str = Field(..., min_length=1, max_length=255)</pre> DESPUÉS (corregido): <pre>min_length=8 + validador de letra y número obligatorios</pre> Impacto: Usuarios podían registrarse con contraseñas de 1 carácter. Fix aplicado: Validador Pydantic @field_validator con regex para complejidad mínima.	
VF-015 ? Puerto de PostgreSQL expuesto al host (0.0.0.0:5435)	MEDIA
OWASP: API8:2023 ? Security Misconfiguration Componente: docker-compose.yml ANTES (vulnerable): <pre>ports: - '5435:5432'</pre> DESPUÉS (corregido): <pre>Puerto eliminado; DB solo accesible dentro de la red Docker</pre> Impacto: DB accesible desde cualquier interfaz del host, ampliando superficie de ataque. Fix aplicado: Puerto comentado; se documenta alternativa loopback 127.0.0.1 para desarrollo.	

3. HALLAZGOS DETALLADOS (MEDIAS)

VF-016 ? APP_KEY de Laravel hardcodeda en docker-compose.yml	MEDIA
OWASP: API8:2023 ? Security Misconfiguration Componente: docker-compose.yml ANTES (vulnerable): <pre>APP_KEY=base64:9KM+d724Xbh0... (fija en el archivo)</pre> DESPUÉS (corregido): <pre>APP_KEY=\${APP_KEY} desde .env</pre> Impacto: Clave de aplicación expuesta; riesgo de descriptado de datos Laravel. Fix aplicado: Externalizada a variable de entorno con instrucción de generación en .env.example.	
VF-017 ? APP_DEBUG=true en producción (Laravel)	MEDIA
OWASP: API8:2023 ? Security Misconfiguration Componente: docker-compose.yml / Flask ANTES (vulnerable): <pre>APP_DEBUG=true en el servicio Laravel</pre> DESPUÉS (corregido): <pre>APP_DEBUG=false</pre> Impacto: Stack traces y detalles de configuración expuestos en respuestas de error. Fix aplicado: Cambiado a false en el docker-compose.yml.	

4. ARQUITECTURA DE SEGURIDAD POST-HARDENING

Flujo de autenticación JWT implementado:

```
[Cliente / Laravel / Flask]
|
| POST /auth/login {username, password}
v
[FastAPI :8000]
| 1. Valida credenciales contra PostgreSQL (bcrypt)
| 2. Genera JWT HS256 {sub: user_id, is_admin, exp: +60min}
| 3. Retorna {access_token, token_type: 'bearer', user}
v
[Cliente recibe token]
|
| GET/POST /endpoint Authorization: Bearer <JWT>
v
[FastAPI ? get_current_user()]
| 1. Extrae token del header Authorization
| 2. Verifica firma HS256 con JWT_SECRET_KEY
| 3. Verifica expiración (exp claim)
| 4. Retorna payload {sub, is_admin}
v
[require_admin()] ? solo para endpoints de gestión
| Verifica is_admin == True en el payload JWT
v
[Endpoint ejecuta lógica de negocio]
```

Controles implementados por capa:

? Red / Docker

- ? PostgreSQL sin puerto expuesto al host (solo red Docker interna)
- ? Redis con contraseña --requirepass y sin puerto expuesto al host
- ? Todos los servicios en red privada visioflow-net

? Aplicación ? FastAPI

- ? JWT Bearer tokens (python-jose, HS256, 60 min TTL)
- ? Rate limiting: 100 req/min por IP (slowapi)
- ? CORS: lista blanca explícita de orígenes
- ? Documentación /docs deshabilitada en producción (APP_ENV)
- ? Validación de contraseñas: mín. 8 chars, letra + número

? Aplicación ? Flask

- ? SECRET_KEY obligatoria desde env (error fatal si no definida)
- ? Cookies de sesión: HttpOnly=True, SameSite=Lax, Secure=True en prod

? Secretos / Configuración

- ? Todas las credenciales en .env (no en docker-compose.yml)
- ? .env.example documenta las variables requeridas
- ? APP_DEBUG=false en producción (Laravel)

5. CONCLUSIONES Y RECOMENDACIONES FUTURAS

5.1 Estado antes del análisis

El sistema Visio Flow en su estado original presentaba 10 vulnerabilidades críticas y 7 medias. La vulnerabilidad más severa era la autenticación basada en un header HTTP (X-Is-Admin: true) que cualquier cliente podía falsificar, combinada con la ausencia total de tokens JWT.

Esto implicaba que un atacante con acceso a la red podía:

- ? Insertar eventos falsos en la base de datos
- ? Leer todos los datos de heatmap y dashboard
- ? Crear/eliminar áreas y usuarios
- ? Acceder a Redis sin autenticación y manipular sesiones
- ? Conectarse directamente a PostgreSQL (puerto 5435 expuesto) con contraseña '123456'

5.2 Estado después del análisis

Tras las implementaciones, el sistema cuenta con:

- ? Autenticación JWT obligatoria en todos los endpoints sensibles
- ? Verificación criptográfica de roles (is_admin en JWT payload)
- ? Rate limiting activo (100 req/min por IP)
- ? CORS configurado con lista blanca
- ? Secretos externalizados en variables de entorno
- ? Base de datos y Redis no accesibles desde el exterior
- ? Contraseñas con complejidad mínima validada
- ? Debug deshabilitado en producción

5.3 Recomendaciones futuras (fuera del alcance actual)

- ? Implementar refresh tokens para renovación sin re-login
- ? Agregar rate limiting específico más estricto en /auth/login (ej. 5 req/min)
- ? Implementar HTTPS/TLS con certificados (nginx reverse proxy)
- ? Agregar logging de auditoría para intentos de login fallidos
- ? Implementar Content Security Policy (CSP) en el dashboard Laravel
- ? Realizar análisis de dependencias (OWASP Dependency Check)
- ? Considerar secrets management dedicado (HashiCorp Vault o similar)
- ? Agregar pruebas de seguridad automatizadas al pipeline CI/CD

RESULTADO FINAL

17 vulnerabilidades identificadas ? 17 corregidas

Reducción de superficie de ataque: SIGNIFICATIVA